



Память в Java: Heap vs Stack простыми словами

Полный текст для озвучки презентации



Слайд 1. Память в Java: Heap vs Stack

«Привет! В этом видео мы разберём одну из самых важных тем в Java — работу с памятью.

Если вы изучаете Java, готовитесь к собеседованию или просто хотите лучше понимать, что происходит внутри JVM, вам обязательно нужно разобраться с двумя понятиями: Stack и Heap.

На первый взгляд всё кажется довольно простым. Мы создаём переменную, вызываем метод, создаём объект через `new` — и программа работает.

Но возникает вопрос: а где именно Java хранит все эти данные?

Например, если мы пишем `int age = 25`, где находится число 25?

А если пишем `Person user = new Person("Alex")`, где находится сам объект Person, а где находится переменная `user`?

И самое интересное — почему неправильная работа с памятью может привести к ошибкам `StackOverflowError` и `OutOfMemoryError`?

Сегодня разберём всё это пошагово и максимально простыми словами.

Главная идея, которую мы должны понять к концу видео:

Stack связан с выполнением методов, а Heap — с объектами.

Погнали!»



Слайд 2. Зачем вообще разбираться в памяти?

«Для начала давайте поймём, зачем обычному Java-разработчику вообще знать, что происходит с памятью.

Мы можем писать Java-код и вообще не думать о памяти.

Написали переменную — Java сама всё организовала.

Создали объект — Java выделила память.

Объект больше не нужен — Garbage Collector когда-нибудь освободит память.

То есть Java действительно многое делает за нас.

Но проблема появляется тогда, когда приложение становится сложнее.

Например, у нас может появиться бесконечная рекурсия, огромное количество объектов или утечка памяти.

И тогда мы начинаем получать ошибки, которые невозможно нормально понять без базового представления о памяти JVM.

Кроме того, вопросы про Stack, Heap и Garbage Collector очень часто встречаются на технических собеседованиях.

Поэтому понимание памяти помогает сразу в нескольких направлениях.

Мы лучше понимаем ошибки программы, можем находить проблемы с памятью и можем уверенно объяснять внутреннюю работу Java.

Но здесь есть важный момент.

То, что мы будем рассматривать дальше, — это в первую очередь удобная концептуальная модель.

Реальная JVM гораздо сложнее и может выполнять различные оптимизации.

Для начала нам достаточно понимать основную логику: методы, Stack Frame, ссылки, объекты, Heap и Garbage Collector.»

Слайд 3. Пример кода

«Давайте начнём с очень простого примера.

У нас есть две строки:

```
int age = 25;
```

и

```
Person user = new Person("Alex");
```

Выглядят они похоже — в обоих случаях мы создаём переменную.

Но внутри JVM это две совершенно разные ситуации.

В первой строке мы создаём локальную переменную типа `int`.

В ней находится значение 25.

Во второй строке появляется переменная `user`.

Но `user` — это не сам объект `Person`.

Это ссылка, с помощью которой мы можем обратиться к объекту.

А объект создаётся благодаря ключевому слову `new`.

То есть здесь у нас можно условно выделить две части.

Первая — переменная `user` и ссылка на объект.

Вторая — сам объект `Person`.

Для простоты мы будем представлять это так:

Stack содержит локальный контекст выполнения и ссылку на объект.

Heap содержит сам объект.

Получается такая цепочка:

Stack → ссылка → Heap → объект.

Именно эту связь мы сегодня будем постепенно разбирать.»

Слайд 4. Что такое Stack?

«Теперь перейдём к Stack.

Stack по-русски обычно называют стеком.

Представьте стопку тарелок.

Когда вы хотите поставить новую тарелку, вы кладёте её сверху.

Когда хотите взять тарелку, тоже берёте верхнюю.

То есть работает принцип LIFO — Last In, First Out.

Последним положили — первым достали.

В Java Stack связан прежде всего с выполнением методов.

Когда программа запускается и начинается выполнение метода `main`, для него создаётся Stack Frame.

Теперь представим, что внутри `main` мы вызываем другой метод — например, `calculate`.

Для `calculate` появляется ещё один Stack Frame.

Если внутри `calculate` мы вызываем `print`, появляется ещё один Frame.

Получается примерно такая структура:

```
main
потом calculate
потом print.
```

Каждый новый вызов метода добавляет новый кадр.

А когда метод заканчивает выполнение, его Stack Frame больше не нужен и удаляется.

Именно поэтому Stack очень тесно связан с жизненным циклом методов.

Ещё один важный момент.

У каждого потока выполнения — Thread — есть собственный Stack.

То есть два разных потока не используют один и тот же Stack для своих вызовов методов.»

Слайд 5. Что хранится в Stack Frame?

«Теперь возникает следующий вопрос.

А что конкретно находится внутри Stack Frame?

Для упрощённого понимания можно сказать, что там находится информация, необходимая для выполнения конкретного метода.

Например, локальные переменные.

Представим такой код:

```
int age = 25;
```

Здесь у нас есть локальная переменная `age` со значением 25.

Также могут быть другие локальные переменные:

```
boolean active = true
```

или

```
double price = 99.9.
```

Кроме того, в Stack Frame могут находиться ссылки на объекты.

Например:

```
Person user = new Person("Alex");
```

Здесь очень важно не перепутать ссылку с самим объектом.

Переменная `user` позволяет нам обратиться к объекту.

Но сам объект `Person` в нашей упрощённой модели находится в Heap.

Поэтому можно представить:

в Stack Frame находится переменная `user` со ссылкой,

а в Heap находится сам объект `Person`.

Именно поэтому на собеседовании важно не говорить просто: "переменная `user` находится в Stack, а объект находится в Heap" без уточнений.

Лучше понимать саму связь: Stack Frame содержит локальный контекст выполнения, а ссылка позволяет обратиться к объекту.»

Слайд 6. Что такое Heap?

«Теперь переходим ко второй важной части — Heap.

Если Stack мы сравнивали со стопкой тарелок, то Heap можно представить как большой склад.

На этом складе размещаются объекты и массивы.

Например:

```
Person user = new Person("Alex");
```

Когда мы используем `new`, создаётся экземпляр класса `Person`.

В нашей упрощённой модели сам объект размещается в Heap.

А переменная `user` содержит ссылку, с помощью которой программа может обратиться к этому объекту.

Можно представить это следующим образом.

Слева у нас Stack.

В нём находится:

```
user → ссылка
```

А справа находится Heap.

В нём:

```
Person Object
```

и внутри объекта могут находиться его поля.

Например:

```
name = "Alex".
```

Ещё один важный момент.

Heap является общей областью памяти для потоков приложения.

То есть объекты, находящиеся в Heap, потенциально могут быть доступны из разных потоков, если на них существуют соответствующие ссылки.

Именно поэтому при работе с многопоточностью нужно учитывать совместный доступ к объектам и состояние этих объектов.»

Слайд 7. Stack и Heap: как они работают вместе?

«Теперь самое главное — объединим Stack и Heap.

Рассмотрим две строки:

```
int age = 25;
```

```
Person user = new Person("Alex");
```

У нас есть Stack Frame текущего метода.

В нём находится локальная переменная `age` со значением 25.

Там же находится переменная `user`, которая содержит ссылку.

А сама ссылка указывает на объект `Person`, расположенный в `Heap`.

То есть визуально это можно представить так:

Stack:

```
age = 25
```

```
user → ссылка
```

И эта ссылка ведёт в `Heap`:

```
Person
```

```
name = "Alex"
```

Вот почему важно понимать разницу между объектом и ссылкой.

Когда мы пишем:

```
user.name
```

мы используем ссылку `user`, чтобы обратиться к объекту `Person` и получить его поле `name`.

И если две разные переменные имеют ссылки на один и тот же объект, они могут обращаться к одному объекту.

Например:

```
Person user1 = new Person("Alex");
```

```
Person user2 = user1;
```

Теперь `user1` и `user2` могут ссылаться на один и тот же объект.

То есть ссылок может быть несколько, а объект при этом один.

Это одна из ключевых идей для понимания Java.»

Слайд 8. Что делает `new`?

«Теперь отдельно разберём ключевое слово `new`.

Возьмём:

```
Person user = new Person("Alex");
```

Что здесь происходит?

Во-первых, JVM должна создать новый экземпляр класса `Person`.

Во-вторых, вызывается конструктор.

Конструктор выполняет необходимую инициализацию объекта.

После создания объекта программа получает ссылку на этот объект.

И эта ссылка записывается в переменную `user`.

То есть очень упрощённо можно представить процесс так:

`new` — создаём объект.

Получаем ссылку.

Записываем ссылку в `user`.

После этого мы можем использовать `user` для доступа к объекту.

Например:

```
user.getName()
```

или

```
user.setName("Alex");
```

Самое важное здесь:

переменная `user` и объект `Person` — это разные сущности.

`user` — это переменная, содержащая ссылку.

`Person` — это объект, на который эта ссылка указывает.

Если ссылок на объект становится несколько, объект при этом не копируется автоматически.

Несколько ссылок могут указывать на один объект.»

Слайд 9. Garbage Collector

«Теперь возникает логичный вопрос.

А что происходит, когда объект нам больше не нужен?

Представим:

```
Person user = new Person("Alex");
```

Пока переменная `user` позволяет добраться до объекта, этот объект считается достижимым.

Теперь мы делаем:

```
user = null;
```

Если других доступных ссылок на этот объект нет, программа больше не может добраться до него.

Такой объект становится недостижимым.

И здесь появляется Garbage Collector — сборщик мусора.

Его задача — находить объекты, которые больше не нужны программе, и освобождать память, которую они занимают.

Но есть очень важное уточнение.

Garbage Collector не обязан запускаться сразу после того, как объект стал недостижимым.

Мы не можем написать:

"Я сделал `user = null`, значит объект прямо сейчас удалён".

Нет.

Объект становится кандидатом для сборки мусора, а конкретный момент освобождения памяти зависит от работы JVM и выбранного алгоритма сборки мусора.

Именно поэтому Garbage Collector — это автоматический механизм управления памятью, но не кнопка мгновенного удаления объектов.»

Слайд 10. StackOverflowError

«Теперь посмотрим на первую классическую ошибку.

`StackOverflowError`.

Название здесь очень говорящее.

Stack — это стек.

Overflow — переполнение.

То есть Stack переполнен.

Посмотрим на пример:

```
void test() {
```

```
    test();
```

```
}
```

Что здесь происходит?

Метод `test` вызывает сам себя.

Но внутри нет никакого условия, которое остановит рекурсию.

Получается:

```
test
```

вызывает `test`.

Новый `test` вызывает ещё один `test`.

И так продолжается снова и снова.

Каждый вызов метода требует нового Stack Frame.

Получается примерно:

```
test()
```

```
test()
```

```
test()
```

```
test()
```

и так далее.

Количество кадров постоянно увеличивается.

Но Stack не бесконечный.

У него есть ограниченный размер.

В какой-то момент JVM уже не может разместить очередной Stack Frame.

И тогда возникает:

```
java.lang.StackOverflowError.
```

Чаще всего мы встречаем эту ошибку при неправильной рекурсии.

Например, когда забыли условие выхода или оно никогда не становится истинным.»

Слайд 11. OutOfMemoryError

«Теперь посмотрим на другую ошибку:

```
OutOfMemoryError.
```

Например:

```
List<byte[]> data = new ArrayList<>();
```

А дальше:

```
while (true) {
```

```
    data.add(new byte[10_000_000]);
```

```
}
```

Что делает программа?

Она бесконечно создаёт большие массивы байтов.

Каждый новый массив занимает память.

Но самое главное — мы сохраняем ссылки на эти массивы внутри списка `data`.

Поэтому эти объекты остаются достижимыми.

Garbage Collector видит, что программа всё ещё может обратиться к этим массивам через список.

Следовательно, он не может просто удалить их как ненужные.

Массив за массивом память Heap постепенно заполняется.

В какой-то момент JVM уже не может выделить достаточно памяти для очередного объекта.

И появляется:

```
java.lang.OutOfMemoryError: Java heap space
```

Здесь мы видим принципиальную разницу.

В случае StackOverflow мы создавали слишком много Stack Frame.

В случае OutOfMemory мы занимали слишком много памяти объектами и другими данными, которые оставались достижимыми.»

Слайд 12. StackOverflow vs OutOfMemory

«Давайте теперь сравним две ошибки.

Первая:

```
StackOverflowError
```

Она связана с переполнением Stack.

Классический пример — бесконечная рекурсия.

Метод вызывает сам себя.

Количество Stack Frame растёт.

Stack заканчивается.

И получаем StackOverflowError.

Вторая:

```
OutOfMemoryError
```

Здесь проблема заключается в том, что JVM не может выделить необходимую память.

В нашем примере мы рассматривали именно переполнение Heap.

Программа создаёт огромное количество объектов и сохраняет ссылки на них.

Garbage Collector не может удалить эти объекты, потому что они всё ещё достижимы.

Heap постепенно заполняется.

И мы получаем `OutOfMemoryError`.

Поэтому для собеседования удобно запомнить простую связь:

`StackOverflowError` — думайте о Stack и слишком большом количестве вызовов.

`OutOfMemoryError` — думайте о нехватке памяти, например Heap.

Но важно помнить, что `OutOfMemoryError` — более общий класс ошибки нехватки памяти и не всегда означает исключительно проблему Heap.»

Слайд 13. Stack vs Heap — сравнение

«Теперь соберём всё в одну таблицу.

Stack — это память, связанная с конкретным потоком выполнения.

У каждого Thread есть собственный Stack.

Stack используется для Stack Frame и локального контекста выполнения методов.

Heap — это общая область памяти JVM, в которой размещаются объекты и массивы в нашей базовой модели.

Жизненный цикл Stack Frame тесно связан с методом.

Метод завершился — его Frame больше не нужен.

С объектами всё немного иначе.

Объект может продолжать существовать после завершения метода, если на него существуют другие доступные ссылки.

Например, метод создал объект и передал его другому объекту или вернул его вызывающему коду.

В этом случае объект продолжает быть достижимым.

Когда объект становится недостижимым, Garbage Collector может освободить его память.

И наконец, типичные ошибки.

Для Stack мы часто вспоминаем `StackOverflowError`.

Для Heap — `OutOfMemoryError`.

Итак:

Stack — выполнение методов.

Heap — объекты.

Reference — связь между ними.»

Слайд 14. Частые ошибки в объяснениях

«Теперь поговорим о нескольких популярных ошибках, которые делают начинающие разработчики.

Первая ошибка:

"Все примитивы всегда находятся в Stack".

Для учебной модели это удобное объяснение, но реальная JVM может выполнять оптимизации.

Например, JIT-компилятор может оптимизировать код и размещение данных.

Вторая ошибка:

"Все объекты всегда находятся только в Heap".

Для базового объяснения мы именно так и говорим.

Но JVM — это не просто набор правил Stack против Heap.

Внутри JVM существуют различные оптимизации, escape analysis, scalar replacement и другие механизмы.

Третья ошибка:

"Если сделать `user = null`, Garbage Collector сразу удалит объект".

Это тоже неправильно.

Объект становится недостижимым и может быть собран GC, но момент запуска сборщика мусора заранее гарантировать нельзя.

И ещё одна популярная фраза:

"Stack всегда быстрее Heap".

В общем объяснении можно сказать, что работа с Stack Frame обычно проще и имеет очень предсказуемую структуру.

Но нельзя превращать это в абсолютное правило для любой операции и любой JVM.

Поэтому на собеседовании лучше говорить аккуратно:

это упрощённая модель работы памяти JVM, предназначенная для понимания основных принципов.»

Слайд 15. Как отвечать на собеседовании?

«Представим, что интервьюер задаёт вопрос:

"Расскажите, чем Stack отличается от Heap в Java?"

Можно ответить следующим образом.

Stack и Heap — важные части модели памяти JVM.

У каждого потока есть собственный Stack, связанный с выполнением методов.

Когда вызывается метод, создаётся Stack Frame.

В нём находится локальный контекст выполнения, включая локальные данные и ссылки.

Heap является общей областью памяти приложения, где в базовой модели размещаются объекты и массивы.

Жизненным циклом объектов управляет Garbage Collector.

Если происходит слишком глубокая или бесконечная рекурсия, Stack может переполниться и возникнет `StackOverflowError`.

Если приложению не хватает памяти для размещения данных, например объектов в Heap, может возникнуть `OutOfMemoryError`.

И если вы хотите показать хороший уровень понимания, можно добавить:

"При этом Stack и Heap — это упрощённая концептуальная модель. Реальная JVM может оптимизировать размещение и выполнение кода".

Такой ответ показывает, что вы понимаете не только базовую теорию, но и ограничения этой модели.»

Слайд 16. Практическая демонстрация в IntelliJ IDEA

«Теперь давайте не просто поговорим о теории, а посмотрим на эти ошибки непосредственно в IntelliJ IDEA.

Сначала продемонстрируем StackOverflow.

Создадим метод:

```
triggerStackOverflow.
```

Внутри он будет вызывать сам себя.

Запускаем программу.

Метод начинает вызывать сам себя снова и снова.

В консоли появляется Stack Trace.

И в конце мы видим:

```
java.lang.StackOverflowError.
```

Теперь сделаем второй эксперимент.

Создадим список:

```
List<byte[]> list = new ArrayList<>();
```

И в бесконечном цикле будем добавлять туда большие массивы.

Например:

```
new byte[10_000_000].
```

Запускаем программу.

Память постепенно заполняется.

Поскольку список сохраняет ссылки на массивы, Garbage Collector не может просто удалить их.

Через некоторое время получаем:

```
java.lang.OutOfMemoryError: Java heap space.
```

Таким образом, мы можем буквально увидеть на практике разницу между двумя проблемами.

В первом эксперименте мы переполняем Stack большим количеством вызовов.

Во втором — заставляем Heap заполниться объектами, которые всё ещё нужны программе с точки зрения достижимости.»

Слайд 17. Финальная шпаргалка

«Давайте теперь соберём весь материал в несколько простых правил.

Первое:

Метод → Stack Frame.

Когда вызывается метод, JVM создаёт соответствующий контекст выполнения.

Второе:

Reference → ссылка на объект.

Переменная может хранить ссылку, которая позволяет обратиться к объекту.

Третье:

new → создание объекта.

Когда мы используем `new`, создаётся новый экземпляр класса.

В нашей базовой модели сам объект размещается в Heap.

И четвертое:

StackOverflowError против OutOfMemoryError.

StackOverflow — часто результат слишком глубокой или бесконечной рекурсии.

OutOfMemory — нехватка памяти, например из-за слишком большого количества объектов, которые остаются достижимыми.

Если нужно запомнить только одну фразу из всего видео, запомните:

Stack хранит контекст выполнения, Heap хранит объекты, а ссылки соединяют их.

Этой модели уже достаточно, чтобы понимать большую часть базовых вопросов о памяти Java.»

Слайд 18. Спасибо за просмотр!

«На этом наше небольшое погружение в память Java заканчивается.

Сегодня мы разобрали, что такое Stack и Heap, что такое Stack Frame, как ссылки связывают переменные с объектами и зачем нужен Garbage Collector.

Мы также разобрали две очень важные ошибки:

`StackOverflowError`

и

`OutOfMemoryError`.

Главное, что нужно унести из этого видео:

Stack — это контекст выполнения методов.

Heap — это область, где в нашей базовой модели находятся объекты и массивы.

Reference — это связь между переменной и объектом.

A Garbage Collector автоматически управляет памятью объектов, которые больше недостижимы.

Если вы изучаете Java, попробуйте не просто запомнить эти определения.

Откройте IntelliJ IDEA, создайте несколько классов, поэкспериментируйте с объектами, рекурсией и коллекциями.

Когда вы увидите эти процессы своими глазами, Stack и Heap перестанут быть абстрактными терминами из учебника.

Спасибо за просмотр!

Удачи в изучении Java и до встречи в следующих видео! 🚀»